23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

O(1): ICPC Practice Tracker

SQL Query Reference

Every SQL statement used in the project, with its purpose, the database concepts it shows, and its output

Roll No	Student Name
AM.SC.U4CSE25245	Saharsh Baiju
AM.SC.U4CSE25244	Rishik Raj P
AM.SC.U4CSE25212	Dev Narayan U
AM.SC.U4CSE25227	Kashyap S

How to read this document

This document lists every SQL statement in O(1): the schema (DDL), the trigger and views, every query the Flask back end runs, extra analysis queries, the constraint tests, and the maintenance commands. Statements are copied from the source files named in each section.

- **Placeholders.** In back-end queries, `%s` and `%(name)s` are psycopg2 parameters. The driver sends the values separately from the SQL text, so user input can never change the query (no SQL injection). `%%` is a literal % sign.
- **Run with.** Each query was executed once against the demo database (4 students, 26,944 submissions, 9,591 problems) with the parameter values shown. Statements that change data were run inside a transaction and rolled back, so the demo data is unchanged.
- **Output.** Copied from PostgreSQL 18.6. Long results show only the first rows; the total row count is noted.

Contents

Part	Contents	Statements
A	Schema: tables, index, trigger and views	A1–A13
B1	Authentication and accounts	1–13
B2	Problems and topics	14–19
B3	Submissions and Codeforces sync	20–24
B4	Practice sheets	25–32
B5	Teams	33–44
B6	Dashboard	45–52
B7	Leaderboard	53–53
B8	Database page	54–55
C	Analysis queries (joins, grouping, subqueries, set operations)	C1–C13
D	Constraint and integrity tests	D1–D20
E	Maintenance: create, upgrade, reset and clear the database	E1–E4
F	Index of SQL concepts	

Total: 13 schema statements, 55 application queries, 13 analysis queries, 20 tests and 4 maintenance commands.

A. Schema (docs/lite/schema-lite.sql)

The whole schema runs as one transaction (BEGIN ... COMMIT), so it is created completely or not at all.

A1. Enable the trigram extension

Adds similarity() and word_similarity() for fuzzy text search.

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

A2. university

Strong entity. univ_name is a second candidate key (UNIQUE).

```
CREATE TABLE university (
  univ_id    SERIAL       PRIMARY KEY,
  univ_name  VARCHAR(120) NOT NULL UNIQUE,
  city       VARCHAR(60)  NOT NULL
);
```

A3. Trigram index on university names

A GIN index over 3-letter chunks of each name; makes the "did you mean" search fast.

```
CREATE INDEX university_name_trgm ON university USING gin (univ_name gin_trgm_ops);
```

A4. team

Strong entity with a unique name.

```
CREATE TABLE team (
  team_id    SERIAL      PRIMARY KEY,
  team_name  VARCHAR(60) NOT NULL UNIQUE
);
```

A5. app_user

Strong entity. email is a candidate key; univ_id is a mandatory FK (total participation); team_id is a nullable FK (partial participation) that becomes NULL when the team is deleted.

```
CREATE TABLE app_user (
  user_id        SERIAL       PRIMARY KEY,
  name           VARCHAR(80)  NOT NULL,
  email          VARCHAR(254) NOT NULL UNIQUE,          -- alternate (candidate) key
  password_hash  VARCHAR(255) NOT NULL,                 -- werkzeug scrypt hash, never the password
  cf_handle      VARCHAR(40)  UNIQUE,                   -- Codeforces handle, optional
  cf_rating      INT,                                   -- current Codeforces rating, refreshed on sync
  cf_max_rating  INT,                                   -- best Codeforces rating
  univ_id        INT          NOT NULL REFERENCES university(univ_id),
  team_id        INT          REFERENCES team(team_id) ON DELETE SET NULL   -- NULL = no team
);
```

A6. topic

Lookup table created during 1NF so each topic name is stored once.

```
CREATE TABLE topic (
  topic_id    SERIAL      PRIMARY KEY,
  topic_name  VARCHAR(40) NOT NULL UNIQUE
);
```

A7. problem

Keyed by the Codeforces code. The CHECK limits rating to Codeforces' range; NULL (unrated) passes a CHECK.

```
CREATE TABLE problem (
  problem_code  VARCHAR(12)  PRIMARY KEY,
  title         VARCHAR(200) NOT NULL,
  rating        INT          CHECK (rating BETWEEN 800 AND 3500)
);
```

A8. problem_topic

Junction table for the M:N relationship tagged. All-key relation.

```
CREATE TABLE problem_topic (
  problem_code  VARCHAR(12) REFERENCES problem(problem_code) ON DELETE CASCADE,
  topic_id      INT         REFERENCES topic(topic_id),
  PRIMARY KEY (problem_code, topic_id)
);
```

A9. submission

Weak entity: key = owner user + problem + partial key submitted_at. CHECK restricts verdicts.

```
CREATE TABLE submission (
  user_id       INT         REFERENCES app_user(user_id) ON DELETE CASCADE,
  problem_code  VARCHAR(12) REFERENCES problem(problem_code),
  submitted_at  TIMESTAMP   NOT NULL,
  verdict       VARCHAR(25) NOT NULL CHECK (verdict IN
                  ('OK', 'WRONG_ANSWER', 'TIME_LIMIT_EXCEEDED', 'RUNTIME_ERROR',
                   'MEMORY_LIMIT_EXCEEDED', 'COMPILATION_ERROR')),
  language      VARCHAR(50) NOT NULL,
  PRIMARY KEY (user_id, problem_code, submitted_at)
);
```

A10. sheet

A practice list owned by one team; deleted with its team.

```
CREATE TABLE sheet (
  sheet_id    SERIAL       PRIMARY KEY,
  title       VARCHAR(120) NOT NULL,
  team_id     INT          NOT NULL REFERENCES team(team_id) ON DELETE CASCADE,
  deadline    DATE
);
```

A11. sheet_problem

Junction table for contains, with the relationship attribute position. Two candidate keys.

```
CREATE TABLE sheet_problem (
  sheet_id      INT         REFERENCES sheet(sheet_id) ON DELETE CASCADE,
  problem_code  VARCHAR(12) REFERENCES problem(problem_code),
  position      SMALLINT    NOT NULL CHECK (position > 0),
  PRIMARY KEY (sheet_id, problem_code),
  UNIQUE (sheet_id, position)                          -- second candidate key
);
```

A12. Trigger: at most three members per team

A rule across rows cannot be a CHECK constraint, so a BEFORE trigger counts the other members and raises an error.

```
CREATE FUNCTION check_team_size() RETURNS trigger AS $$
BEGIN
  IF NEW.team_id IS NOT NULL AND
     (SELECT count(*) FROM app_user
       WHERE team_id = NEW.team_id AND user_id <> NEW.user_id) >= 3 THEN
    RAISE EXCEPTION 'team % already has 3 members', NEW.team_id;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER team_size_check
  BEFORE INSERT OR UPDATE OF team_id ON app_user
  FOR EACH ROW EXECUTE FUNCTION check_team_size();
```

A13. Views for derived values

Difficulty, solved counts and sheet progress are computed, never stored, so they can never contradict the submissions.

```
CREATE VIEW v_problem_difficulty AS
SELECT problem_code, title, rating,
       CASE WHEN rating IS NULL THEN NULL          -- unrated (new) problem
            WHEN rating < 1400 THEN 'Easy'
            WHEN rating < 2000 THEN 'Medium'
            ELSE 'Hard' END AS difficulty
FROM problem;

-- Leaderboard: distinct problems solved per user.
CREATE VIEW v_leaderboard AS
SELECT u.user_id, u.name, un.univ_name,
       count(DISTINCT s.problem_code) AS solved
FROM app_user u
JOIN university un ON un.univ_id = u.univ_id
LEFT JOIN submission s ON s.user_id = u.user_id AND s.verdict = 'OK'
GROUP BY u.user_id, u.name, un.univ_name;

-- Sheet progress: for every member of the sheet's team, how many sheet problems are solved.
CREATE VIEW v_sheet_progress AS
SELECT sh.sheet_id, u.user_id, u.name,
       count(sp.problem_code) AS total,
       count(sp.problem_code) FILTER (WHERE EXISTS (
         SELECT 1 FROM submission s
         WHERE s.user_id = u.user_id AND s.problem_code = sp.problem_code
           AND s.verdict = 'OK')) AS solved
FROM sheet sh
JOIN app_user u       ON u.team_id = sh.team_id
JOIN sheet_problem sp ON sp.sheet_id = sh.sheet_id
GROUP BY sh.sheet_id, u.user_id, u.name;
```

Output of the views on the demo data

```
SELECT * FROM v_problem_difficulty ORDER BY rating DESC NULLS LAST LIMIT 5;
```

Output:

problem_code	title	rating	difficulty
1427H	Prison Break	3500	Hard
1578G	Game of Chance	3500	Hard
1446E	Long Recovery	3500	Hard
1470F	Strange Covering	3500	Hard
750H	New Year and Snowy Grid	3500	Hard

```
SELECT * FROM v_leaderboard ORDER BY solved DESC;
```

Output:

user_id	name	univ_name	solved
2	Demo Student B	Amrita Vishwa Vidyapeetham	4596
3	Demo Student C	Amrita Vishwa Vidyapeetham	3956
4	Demo Student D	NIT Calicut	3049
1	Demo Student A	Amrita Vishwa Vidyapeetham	1542

```
SELECT * FROM v_sheet_progress ORDER BY solved DESC;
```

Output:

sheet_id	user_id	name	total	solved
1	3	Demo Student C	8	3
1	1	Demo Student A	8	1
1	2	Demo Student B	8	0

B1. Authentication and accounts

Source: `backend/routes/auth.py`

1. Current user's team

Purpose: Finds the team of the signed-in user. Used by almost every route to scope data to the team.

Concepts: Selection by primary key

```
SELECT team_id FROM app_user WHERE user_id = %s
```

Run with: `(1,)`

Output:

team_id
1

2. Load a user with university and team names

Purpose: Builds the profile sent to the browser. password_hash is deliberately not selected.

Concepts: INNER JOIN, LEFT JOIN (a user may have no team), projection

```
SELECT u.user_id, u.name, u.email, u.cf_handle, u.cf_rating, u.cf_max_rating, u.univ_id, u.team_id,
       un.univ_name, t.team_name
FROM app_user u
JOIN university un ON un.univ_id = u.univ_id
LEFT JOIN team t   ON t.team_id = u.team_id
WHERE u.user_id = %s
```

Run with: `(1,)`

Output:

user_id	name	email	cf_handle	cf_rating	cf_max_rating	univ_id	team_id	univ_name	team_name
1	Demo Student A	a@demo.test	Errichto	2624	3053	1	1	Amrita Vishwa Vidyapeetham	O(1)

3. List universities

Purpose: Fills the college list.

Concepts: ORDER BY

```
SELECT univ_id, univ_name, city FROM university ORDER BY univ_name
```

Run with: `()`

Output:

univ_id	univ_name	city
1	Amrita Vishwa Vidyapeetham	Amritapuri
2	NIT Calicut	Kozhikode

4. "Did you mean" college search

Purpose: Finds colleges whose name is close to what the student typed, even with typos, so the same college is not added twice.

Concepts: pg_trgm similarity() and word_similarity(), GIN trigram index, ILIKE pattern matching, ORDER BY on a boolean expression, LIMIT

```
SELECT univ_id, univ_name, city,
       greatest(similarity(%(q)s, univ_name), word_similarity(%(q)s, univ_name)) AS score
FROM university
WHERE univ_name ILIKE '%%' || %(q)s || '%%'
   OR word_similarity(%(q)s, univ_name) > 0.3
ORDER BY univ_name ILIKE %(q)s || '%%' DESC, score DESC
LIMIT 6
```

Run with: `{'q': 'amritha'}`

Output:

univ_id	univ_name	city	score
1	Amrita Vishwa Vidyapeetham	Amritapuri	0.625

5. Sign-up: does the college exist?

Purpose: Case-insensitive lookup so 'amrita vishwa vidyapeetham' re-uses the existing row.

Concepts: Function in WHERE (lower)

```
SELECT univ_id FROM university WHERE lower(univ_name) = lower(%s)
```

Run with: `('amrita vishwa vidyapeetham',)`

Output:

univ_id
1

6. Sign-up: add a new college

Purpose: Runs only when the college is new. RETURNING gives the generated key without a second query.

Concepts: INSERT ... RETURNING, SERIAL key, UNIQUE(univ_name)

```
INSERT INTO university (univ_name, city) VALUES (%s, %s) RETURNING univ_id
```

Run with: `('IIT Madras', 'Chennai')`

Output:

univ_id
3

7. Sign-up: is the e-mail taken?

Purpose: Friendly error before the UNIQUE constraint fires.

Concepts: Existence check

```
SELECT 1 FROM app_user WHERE email = %s
```

Run with: `('a@demo.test',)`

Output:

?column?
1

8. Sign-up: is the Codeforces handle taken?

Purpose: Handles are unique, ignoring case.

Concepts: Existence check, lower()

```
SELECT 1 FROM app_user WHERE lower(cf_handle) = lower(%s)
```

Run with: `('errichto',)`

Output:

?column?
1

9. Sign-up: create the user

Purpose: Stores the new student with a scrypt password hash (never the password).

Concepts: INSERT ... RETURNING, foreign key univ_id

```
INSERT INTO app_user (name, email, password_hash, cf_handle, univ_id)
VALUES (%s, %s, %s, %s, %s)
RETURNING user_id
```

Run with: `('New Student', 'new@demo.test', 'scrypt:demo-hash', None, 1)`

Output:

user_id
5

10. Log in

Purpose: Fetches the stored hash; the password is checked in Python with check_password_hash.

Concepts: Selection on the alternate key email

```
SELECT user_id, password_hash FROM app_user WHERE email = %s
```

Run with: `('a@demo.test',)`

Output:

user_id	password_hash
1	(hidden)

11. Edit profile

Purpose: Updates the signed-in user. UNIQUE(email) and UNIQUE(cf_handle) are enforced by the database.

Concepts: UPDATE with WHERE on primary key

```
UPDATE app_user SET name = %s, email = %s, cf_handle = %s, univ_id = %s WHERE user_id = %s
```

Run with: `('Demo Student A', 'a@demo.test', 'Errichto', 1, 1)`

Output:

```
UPDATE 1      -- (rolled back after the demo)
```

12. Change password: read current hash

Purpose: Verifies the current password before changing it.

Concepts: Selection

```
SELECT password_hash FROM app_user WHERE user_id = %s
```

Run with: `(1,)`

Output:

password_hash
(hidden)

13. Change password: store new hash

Purpose: Saves the new scrypt hash.

Concepts: UPDATE

```
UPDATE app_user SET password_hash = %s WHERE user_id = %s
```

Run with: `('scrypt:new-demo-hash', 1)`

Output:

```
UPDATE 1      -- (rolled back after the demo)
```

B2. Problems and topics

Source: `backend/routes/problems.py`

14. Insert a problem (if new)

Purpose: Adds a Codeforces problem. If it already exists nothing happens, and RETURNING gives no row.

Concepts: UPSERT: INSERT ... ON CONFLICT DO NOTHING, CHECK (rating BETWEEN 800 AND 3500)

```
INSERT INTO problem (problem_code, title, rating)
VALUES (%s, %s, %s)
ON CONFLICT (problem_code) DO NOTHING
RETURNING problem_code
```

Run with: `('99999Z', 'Demo Problem', 1200)`

Output:

problem_code
99999Z

15. Insert a topic (or get the existing one)

Purpose: A no-op DO UPDATE so RETURNING also returns the id of a topic that already exists.

Concepts: UPSERT: ON CONFLICT DO UPDATE, EXCLUDED pseudo-table, UNIQUE(topic_name)

```
INSERT INTO topic (topic_name) VALUES (%s)
ON CONFLICT (topic_name) DO UPDATE SET topic_name = EXCLUDED.topic_name
RETURNING topic_id
```

Run with: `('dp',)`

Output:

topic_id
1

16. Tag a problem with a topic

Purpose: Fills the M:N junction table.

Concepts: Junction table, composite primary key, ON CONFLICT DO NOTHING

```
INSERT INTO problem_topic (problem_code, topic_id) VALUES (%s, %s) ON CONFLICT DO NOTHING
```

Run with: `('455A', 1)`

Output:

```
INSERT 0 0      -- (rolled back after the demo)
```

17. Problem list with topics, difficulty and my status

Purpose: One row per problem: its topics as an array, its difficulty band from the view, and whether I solved, attempted or have not tried it.

Concepts: JOIN with a VIEW, LEFT JOINs, GROUP BY, array_agg ... FILTER, bool_or aggregate, CASE, COALESCE, NULLS LAST

```
SELECT p.problem_code, p.title, p.rating, d.difficulty,
       COALESCE(array_agg(DISTINCT t.topic_name)
                FILTER (WHERE t.topic_name IS NOT NULL), '{}') AS topics,
       CASE WHEN bool_or(s.verdict = 'OK') THEN 'solved'
            WHEN count(s.verdict) > 0     THEN 'attempted'
            ELSE 'todo' END               AS status
FROM problem p
JOIN v_problem_difficulty d ON d.problem_code = p.problem_code
LEFT JOIN problem_topic pt  ON pt.problem_code = p.problem_code
LEFT JOIN topic t           ON t.topic_id = pt.topic_id
LEFT JOIN submission s      ON s.problem_code = p.problem_code AND s.user_id = %s
GROUP BY p.problem_code, p.title, p.rating, d.difficulty
ORDER BY p.rating NULLS LAST, p.problem_code
```

Run with: `(1,)`

Output:

problem_code	title	rating	difficulty	topics	status
1002A1	Generate superposition of all basis st…	800	Easy	{}	todo
1003A	Polycarp's Pockets	800	Easy	{implementation}	todo
1006A	Adjacent Replacements	800	Easy	{implementation}	todo
1017A	The Rank	800	Easy	{implementation}	todo
1028A	Find Square	800	Easy	{implementation}	todo
1041A	Heist	800	Easy	{greedy,implementation,sortings}	todo

(6 of 9591 rows shown)

18. List topics

Purpose: Fills the topic filter.

Concepts: ORDER BY

```
SELECT topic_id, topic_name FROM topic ORDER BY topic_name
```

Run with: `()`

Output:

topic_id	topic_name
1076	2-sat
9	binary search
33	bitmasks
12	brute force
324	chinese remainder theorem
38	combinatorics

(6 of 37 rows shown)

19. Does a problem exist?

Purpose: Used before adding a problem by hand and before building a sheet.

Concepts: Existence check on primary key

```
SELECT 1 FROM problem WHERE problem_code = %s
```

Run with: `('455A',)`

Output:

?column?
1

B3. Submissions and Codeforces sync

Source: `backend/routes/submissions.py, backend/services/codeforces.py`

20. Submission history (me / team / all, optional verdict)

Purpose: The WHERE clause is built from the filters the user picks; values are always passed as parameters, never pasted into the SQL.

Concepts: 3-table JOIN, dynamic WHERE with parameters, ORDER BY DESC, LIMIT

```
SELECT s.submitted_at, s.user_id, u.name, s.problem_code, p.title, s.language, s.verdict
FROM submission s
JOIN app_user u ON u.user_id = s.user_id
JOIN problem p  ON p.problem_code = s.problem_code
WHERE s.user_id = %s AND s.verdict = %s        -- built from ?scope= and ?verdict=
ORDER BY s.submitted_at DESC
LIMIT 500
```

Run with: `(1, 'OK')`

Output:

submitted_at	user_id	name	problem_code	title	language	verdict
2026-10-10 18:38	1	Demo Student A	2271H1	Personal Issues (Easy Version)	C++23 (GCC 14-64, msys2)	OK
2026-10-10 17:21	1	Demo Student A	2271F	Beautiful Pillers	C++23 (GCC 14-64, msys2)	OK
2026-10-10 16:12	1	Demo Student A	2271E	Array Emptying	C++23 (GCC 14-64, msys2)	OK
2026-10-10 15:49	1	Demo Student A	2271D	Target Infection	C++23 (GCC 14-64, msys2)	OK
2026-10-10 15:28	1	Demo Student A	2271C	XOR Problem	C++23 (GCC 14-64, msys2)	OK
2026-10-10 15:12	1	Demo Student A	2271B	MEX Game	C++23 (GCC 14-64, msys2)	OK

(6 of 500 rows shown)

21. Log one submission by hand

Purpose: Foreign keys, CHECK(verdict IN ...) and the composite primary key are enforced by the database.

Concepts: INSERT into a weak-entity table (composite key)

```
INSERT INTO submission (user_id, problem_code, submitted_at, verdict, language)
VALUES (%s, %s, %s, %s, %s)
```

Run with: `(1, '4A', '2026-10-11 09:00:00', 'WRONG_ANSWER', 'C++17')`

Output:

```
INSERT 0 1      -- (rolled back after the demo)
```

22. Read my Codeforces handle

Purpose: Sync needs the handle.

Concepts: Selection

```
SELECT cf_handle FROM app_user WHERE user_id = %s
```

Run with: `(1,)`

Output:

cf_handle
Errichto

23. Store the Codeforces rating

Purpose: Snapshot of the rating, refreshed on every sync.

Concepts: UPDATE

```
UPDATE app_user SET cf_rating = %s, cf_max_rating = %s WHERE user_id = %s
```

Run with: `(2624, 3053, 1)`

Output:

```
UPDATE 1      -- (rolled back after the demo)
```

24. Import a Codeforces submission

Purpose: Re-syncing is safe: a submission already stored is skipped by the primary key, so nothing is duplicated.

Concepts: Idempotent INSERT ... ON CONFLICT (composite key) DO NOTHING RETURNING

```
INSERT INTO submission (user_id, problem_code, submitted_at, verdict, language)
VALUES (%s, %s, %s, %s, %s)
ON CONFLICT (user_id, problem_code, submitted_at) DO NOTHING
RETURNING 1
```

Run with: `(1, '4A', '2026-10-11 09:05:00', 'OK', 'C++17')`

Output:

?column?
1

B4. Practice sheets

Source: `backend/routes/sheets.py`

25. Find a sheet of my team

Purpose: A sheet is visible only to the team that owns it.

Concepts: Selection with two conditions

```
SELECT sheet_id, title, team_id, deadline FROM sheet WHERE sheet_id = %s AND team_id = %s
```

Run with: `(1, 1)`

Output:

sheet_id	title	team_id	deadline
1	Binary Search Week	1	2026-10-20

26. My team's sheets with my progress

Purpose: Progress comes from the v_sheet_progress view, so it is never stored and can never be out of date.

Concepts: LEFT JOIN with a VIEW, COALESCE, NULLS LAST

```
SELECT sh.sheet_id, sh.title, sh.deadline,
       COALESCE(vp.total, 0) AS total, COALESCE(vp.solved, 0) AS solved
FROM sheet sh
LEFT JOIN v_sheet_progress vp ON vp.sheet_id = sh.sheet_id AND vp.user_id = %s
WHERE sh.team_id = %s
ORDER BY sh.deadline NULLS LAST, sh.sheet_id
```

Run with: `(1, 1)`

Output:

sheet_id	title	deadline	total	solved
1	Binary Search Week	2026-10-20	8	1

27. Problems of a sheet, in order

Purpose: Ordered by the position column of the junction table.

Concepts: JOIN junction table + VIEW, ORDER BY

```
SELECT sp.position, p.problem_code, p.title, p.rating, d.difficulty
FROM sheet_problem sp
JOIN problem p              ON p.problem_code = sp.problem_code
JOIN v_problem_difficulty d ON d.problem_code = sp.problem_code
WHERE sp.sheet_id = %s
ORDER BY sp.position
```

Run with: `(1,)`

Output:

position	problem_code	title	rating	difficulty
1	2257D	Bermuda Rectangle	1600	Medium
2	2254E	Chronostasis	1500	Medium
3	2232C2	Seating Arrangement (Hard Version)	1600	Medium
4	2229D	Me When Median	1700	Medium

position	problem_code	title	rating	difficulty
		Problem		
5	2228C2	Cirno and Number (Hard Version)	1600	Medium
6	2228C1	Cirno and Number (Easy Version)	1500	Medium

(6 of 8 rows shown)

28. Team members' progress on a sheet

Purpose: Every member appears even before solving anything (LEFT JOIN).

Concepts: LEFT JOIN with a VIEW

```
SELECT u.user_id, u.name, COALESCE(vp.solved, 0) AS solved, COALESCE(vp.total, 0) AS total
FROM app_user u
LEFT JOIN v_sheet_progress vp ON vp.user_id = u.user_id AND vp.sheet_id = %s
WHERE u.team_id = %s
ORDER BY u.name
```

Run with: `(1, 1)`

Output:

user_id	name	solved	total
1	Demo Student A	1	8
2	Demo Student B	0	8
3	Demo Student C	3	8

29. Who solved / attempted which sheet problem

Purpose: Fills the tick grid on the Sheets page.

Concepts: JOIN, GROUP BY, bool_or, CASE

```
SELECT s.user_id, s.problem_code,
       CASE WHEN bool_or(s.verdict = 'OK') THEN 'solved' ELSE 'attempted' END AS status
FROM submission s
JOIN sheet_problem sp ON sp.problem_code = s.problem_code AND sp.sheet_id = %s
JOIN app_user u       ON u.user_id = s.user_id AND u.team_id = %s
GROUP BY s.user_id, s.problem_code
```

Run with: `(1, 1)`

Output:

user_id	problem_code	status
1	455A	solved
3	2229D	solved
3	455A	solved
3	4A	solved

30. Create a sheet

Purpose: Header row of the sheet.

Concepts: INSERT ... RETURNING, FK team_id

```
INSERT INTO sheet (title, team_id, deadline) VALUES (%s, %s, %s) RETURNING sheet_id
```

Run with: `('Demo sheet', 1, '2026-10-31')`

Output:

sheet_id
2

31. Add a problem to a sheet

Purpose: One row per problem; UNIQUE(sheet_id, position) keeps positions distinct.

Concepts: Junction table with an attribute (position)

```
INSERT INTO sheet_problem (sheet_id, problem_code, position) VALUES (%s, %s, %s)
```

Run with: `(1, '1A', 99)`

Output:

```
INSERT 0 1      -- (rolled back after the demo)
```

32. Delete a sheet

Purpose: ON DELETE CASCADE removes its sheet_problem rows automatically.

Concepts: DELETE, referential action CASCADE

```
DELETE FROM sheet WHERE sheet_id = %s
```

Run with: `(1,)`

Output:

```
DELETE 1      -- (rolled back after the demo)
```

B5. Teams

Source: `backend/routes/teams.py`

33. All teams with members and solved count

Purpose: Empty teams still appear because of the LEFT JOINs.

Concepts: LEFT JOIN, GROUP BY, count(DISTINCT), string_agg, COALESCE

```
SELECT t.team_id, t.team_name,
       count(DISTINCT u.user_id)                        AS members,
       COALESCE(string_agg(DISTINCT u.name, ', '), '') AS member_names,
       count(DISTINCT s.problem_code)                   AS solved
FROM team t
LEFT JOIN app_user u   ON u.team_id = t.team_id
LEFT JOIN submission s ON s.user_id = u.user_id AND s.verdict = 'OK'
GROUP BY t.team_id, t.team_name
ORDER BY solved DESC, t.team_name
```

Run with: `()`

Output:

team_id	team_name	members	member_names	solved
1	O(1)	3	Demo Student A, Demo Student B, Demo S…	7847

34. My team's name

Purpose: Header of the team page.

Concepts: Selection

```
SELECT team_id, team_name FROM team WHERE team_id = %s
```

Run with: `(1,)`

Output:

team_id	team_name
1	O(1)

35. My team's members

Purpose: Members with their university and solved count from the leaderboard view.

Concepts: JOIN with a VIEW

```
SELECT u.user_id, u.name, u.cf_handle, un.univ_name, lb.solved
FROM app_user u
JOIN university un    ON un.univ_id = u.univ_id
JOIN v_leaderboard lb ON lb.user_id = u.user_id
WHERE u.team_id = %s
ORDER BY u.name
```

Run with: `(1,)`

Output:

user_id	name	cf_handle	univ_name	solved
1	Demo Student A	Errichto	Amrita Vishwa Vidyapeetham	1542
2	Demo Student B	Um_nik	Amrita Vishwa Vidyapeetham	4596

user_id	name	cf_handle	univ_name	solved
3	Demo Student C	Benq	Amrita Vishwa Vidyapeetham	3956

36. Distinct problems solved by the team

Purpose: A problem solved by two members counts once.

Concepts: count(DISTINCT), JOIN

```
SELECT count(DISTINCT s.problem_code) AS n
FROM submission s JOIN app_user u ON u.user_id = s.user_id
WHERE u.team_id = %s AND s.verdict = 'OK'
```

Run with: `(1,)`

Output:

n
7847

37. Number of team sheets

Purpose: Shown on the team card.

Concepts: count(*)

```
SELECT count(*) AS n FROM sheet WHERE team_id = %s
```

Run with: `(1,)`

Output:

n
1

38. Create a team

Purpose: Runs in the same transaction as the next statement, so the team is never created without its first member.

Concepts: INSERT ... RETURNING, transaction (atomicity)

```
INSERT INTO team (team_name) VALUES (%s) RETURNING team_id
```

Run with: `('Demo Team',)`

Output:

team_id
2

39. Join a team

Purpose: The team_size_check trigger rejects a fourth member.

Concepts: UPDATE, BEFORE UPDATE trigger

```
UPDATE app_user SET team_id = %s WHERE user_id = %s
```

Run with: `(1, 4)`

Output:

```
ERROR:  team 1 already has 3 members
```

40. Leave a team

Purpose: Membership is optional, so team_id becomes NULL.

Concepts: UPDATE to NULL (nullable FK)

```
UPDATE app_user SET team_id = NULL WHERE user_id = %s
```

Run with: `(3,)`

Output:

```
UPDATE 1      -- (rolled back after the demo)
```

41. Team analytics: head-to-head numbers

Purpose: Rating, problems solved, solved this month, active days, current and longest streak, acceptance, average and hardest rating, and problems only that member solved. All computed from stored submissions in one query.

Concepts: WITH (CTE), window function row_number() OVER (PARTITION BY ... ORDER BY), gaps-and-islands, correlated scalar subqueries, NOT EXISTS, FILTER, NULLIF, date arithmetic, intervals

```
WITH first_solves AS (
    SELECT s.user_id, s.problem_code, min(s.submitted_at) AS solved_at
    FROM submission s
    JOIN app_user u ON u.user_id = s.user_id
    WHERE u.team_id = %(team)s AND s.verdict = 'OK'
    GROUP BY s.user_id, s.problem_code
),
active_days AS (
    SELECT DISTINCT s.user_id, s.submitted_at::date AS day
    FROM submission s
    JOIN app_user u ON u.user_id = s.user_id
    WHERE u.team_id = %(team)s
),
-- "Gaps and islands": for consecutive days, day - row_number() is the
-- same value, so each group of equal values is one unbroken streak.
islands AS (
    SELECT user_id, count(*) AS length, max(day) AS last_day
    FROM (
        SELECT user_id, day,
               day - row_number() OVER (PARTITION BY user_id ORDER BY day)::int AS island
        FROM active_days
    ) numbered
    GROUP BY user_id, island
)
SELECT u.user_id, u.name, u.cf_handle, u.cf_rating, u.cf_max_rating,
       (SELECT count(*) FROM first_solves f WHERE f.user_id = u.user_id) AS solved,
       (SELECT count(*) FROM first_solves f
         WHERE f.user_id = u.user_id AND f.solved_at > %(now)s - interval '30 days') AS solved_30d,
       (SELECT count(*) FROM active_days a
         WHERE a.user_id = u.user_id AND a.day > %(today)s - 30) AS active_days_30,
       (SELECT COALESCE(max(length), 0) FROM islands i
         WHERE i.user_id = u.user_id AND i.last_day >= %(today)s - 1) AS current_streak,
       (SELECT COALESCE(max(length), 0) FROM islands i WHERE i.user_id = u.user_id) AS longest_streak,
       (SELECT round(100.0 * count(*) FILTER (WHERE s.verdict = 'OK') / NULLIF(count(*), 0))::int
          FROM submission s WHERE s.user_id = u.user_id) AS acceptance,
       (SELECT round(avg(p.rating))::int FROM first_solves f
          JOIN problem p ON p.problem_code = f.problem_code
         WHERE f.user_id = u.user_id) AS avg_solved_rating,
       (SELECT max(p.rating) FROM first_solves f
          JOIN problem p ON p.problem_code = f.problem_code
         WHERE f.user_id = u.user_id) AS hardest_solved,
       -- solved by this member and by no one else in the team
       (SELECT count(*) FROM first_solves f
         WHERE f.user_id = u.user_id
           AND NOT EXISTS (SELECT 1 FROM first_solves g
                           WHERE g.problem_code = f.problem_code AND g.user_id <> f.user_id)) AS unique_solves
FROM app_user u
WHERE u.team_id = %(team)s
ORDER BY u.name
```

Run with: `{'team': 1, 'now': '2026-10-11 15:55', 'today': '2026-10-11'}`

Output:

user_id	name	cf_handle	cf_rating	cf_max_rating	solved	solved_30d	active_days_30	current_streak	longest_streak	acceptance	avg_solved_rating	hardest_solved	unique_solves
1	Demo Student A	Errichto	2624	3053	1542	7	1	1	7	50	1843	3500	676
2	Demo Student B	Um_nik	3353	3663	4596	15	2	1	7	64	1892	3500	2849
3	Demo Student C	Benq	3676	3857	3956	7	2	0	20	56	2142	3500	2396

42. Team analytics: problems solved per week

Purpose: Feeds the 12-week bar charts.

Concepts: CTE, date_trunc, GROUP BY

```
WITH first_solves AS (
    SELECT s.user_id, s.problem_code, min(s.submitted_at) AS solved_at
    FROM submission s
    JOIN app_user u ON u.user_id = s.user_id
    WHERE u.team_id = %(team)s AND s.verdict = 'OK'
    GROUP BY s.user_id, s.problem_code
)
SELECT user_id, date_trunc('week', solved_at)::date AS week, count(*) AS solved
FROM first_solves
WHERE solved_at >= %(since)s
GROUP BY user_id, week
```

Run with: `{'team': 1, 'since': '2026-07-20'}`

Output:

user_id	week	solved
1	2026-08-03	7
1	2026-08-10	3
1	2026-10-05	7
2	2026-09-21	8
2	2026-10-05	7
3	2026-07-20	11

(6 of 12 rows shown)

43. Team analytics: solved per topic

Purpose: Feeds the topic-coverage heatmap.

Concepts: CTE, JOIN through the M:N junction table, GROUP BY

```
WITH first_solves AS (
    SELECT s.user_id, s.problem_code, min(s.submitted_at) AS solved_at
    FROM submission s
    JOIN app_user u ON u.user_id = s.user_id
    WHERE u.team_id = %(team)s AND s.verdict = 'OK'
    GROUP BY s.user_id, s.problem_code
)
SELECT f.user_id, t.topic_name, count(*) AS solved
FROM first_solves f
JOIN problem_topic pt ON pt.problem_code = f.problem_code
JOIN topic t          ON t.topic_id = pt.topic_id
GROUP BY f.user_id, t.topic_name
```

Run with: `{'team': 1}`

Output:

user_id	topic_name	solved
1	greedy	320
3	expression parsing	6
2	combinatorics	274
1	math	368
2	probabilities	95
1	probabilities	36

(6 of 109 rows shown)

44. Members to sync

Purpose: Members with a Codeforces handle.

Concepts: IS NOT NULL

```
SELECT user_id, name, cf_handle FROM app_user WHERE team_id = %s AND cf_handle IS NOT NULL
```

Run with: `(1,)`

Output:

user_id	name	cf_handle
1	Demo Student A	Errichto
2	Demo Student B	Um_nik
3	Demo Student C	Benq

B6. Dashboard

Source: `backend/routes/dashboard.py`

45. My solved count, submissions and acceptance

Purpose: Three numbers from one scan of my submissions.

Concepts: Aggregate FILTER clause, count(DISTINCT), NULLIF to avoid division by zero, COALESCE, round

```
SELECT count(DISTINCT problem_code) FILTER (WHERE verdict = 'OK') AS solved,
       count(*)                                                   AS submissions,
       COALESCE(round(100.0 * count(*) FILTER (WHERE verdict = 'OK')
                      / NULLIF(count(*), 0)), 0)::int             AS acceptance
FROM submission
WHERE user_id = %s
```

Run with: `(1,)`

Output:

solved	submissions	acceptance
1542	3467	50

46. My rank

Purpose: Ranks every student and keeps my row.

Concepts: Window functions rank() OVER and count(*) OVER (), derived table

```
SELECT rank, players FROM (
    SELECT user_id, rank() OVER (ORDER BY solved DESC) AS rank, count(*) OVER () AS players
    FROM v_leaderboard
) ranked
WHERE user_id = %s
```

Run with: `(1,)`

Output:

rank	players
4	4

47. Problems in the database

Purpose: Shown under the solved count.

Concepts: count(*)

```
SELECT count(*) AS n FROM problem
```

Run with: `()`

Output:

n
9591

48. Up next: sheet problems I have not solved

Purpose: The next five problems to practise.

Concepts: NOT EXISTS correlated subquery, 3-table JOIN, LIMIT

```
SELECT sh.sheet_id, sh.title AS sheet_title, sp.position, p.problem_code, p.title, p.rating
FROM sheet sh
JOIN sheet_problem sp ON sp.sheet_id = sh.sheet_id
JOIN problem p        ON p.problem_code = sp.problem_code
WHERE sh.team_id = %s
  AND NOT EXISTS (SELECT 1 FROM submission s
                  WHERE s.user_id = %s AND s.problem_code = sp.problem_code
                    AND s.verdict = 'OK')
ORDER BY sh.deadline NULLS LAST, sp.position
LIMIT 5
```

Run with: `(1, 1)`

Output:

sheet_id	sheet_title	position	problem_code	title	rating
1	Binary Search Week	1	2257D	Bermuda Rectangle	1600
1	Binary Search Week	2	2254E	Chronostasis	1500
1	Binary Search Week	3	2232C2	Seating Arrangement (Hard Version)	1600
1	Binary Search Week	4	2229D	Me When Median Problem	1700
1	Binary Search Week	5	2228C2	Cirno and Number (Hard Version)	1600

49. My recent submissions

Purpose: Last six attempts.

Concepts: JOIN, ORDER BY DESC, LIMIT

```
SELECT s.submitted_at, s.problem_code, p.title, s.language, s.verdict
FROM submission s JOIN problem p ON p.problem_code = s.problem_code
WHERE s.user_id = %s
ORDER BY s.submitted_at DESC
LIMIT 6
```

Run with: `(1,)`

Output:

submitted_at	problem_code	title	language	verdict
2026-10-10 18:38	2271H1	Personal Issues (Easy Version)	C++23 (GCC 14-64, msys2)	OK
2026-10-10 18:30	2271H1	Personal Issues (Easy Version)	C++23 (GCC 14-64, msys2)	WRONG_ANSWER
2026-10-10 17:47	2271H1	Personal Issues (Easy Version)	C++23 (GCC 14-64, msys2)	WRONG_ANSWER
2026-10-10 17:21	2271F	Beautiful Pillers	C++23 (GCC 14-64, msys2)	OK
2026-10-10 16:12	2271E	Array Emptying	C++23 (GCC 14-64, msys2)	OK
2026-10-10 15:49	2271D	Target Infection	C++23 (GCC 14-64, msys2)	OK

50. Solved by topic

Purpose: For each topic: problems in the database and how many I solved.

Concepts: LEFT JOIN, count(DISTINCT) FILTER, GROUP BY

```
SELECT t.topic_name,
       count(DISTINCT pt.problem_code) AS total,
       count(DISTINCT pt.problem_code) FILTER (WHERE s.problem_code IS NOT NULL) AS solved
FROM topic t
JOIN problem_topic pt  ON pt.topic_id = t.topic_id
LEFT JOIN submission s ON s.problem_code = pt.problem_code AND s.user_id = %s AND s.verdict = 'OK'
GROUP BY t.topic_name
ORDER BY solved DESC, total DESC
LIMIT 10
```

Run with: `(1,)`

Output:

topic_name	total	solved
math	2370	368
greedy	2337	320
implementation	1803	304
dp	1741	251
constructive algorithms	1456	218
brute force	1274	210

(6 of 10 rows shown)

51. Solved by difficulty

Purpose: Easy / Medium / Hard bands come from the view.

Concepts: VIEW, EXISTS inside FILTER, GROUP BY, ORDER BY an aggregate

```
SELECT d.difficulty,
       count(*) AS total,
       count(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM submission s
           WHERE s.problem_code = d.problem_code AND s.user_id = %s AND s.verdict = 'OK')) AS solved
FROM v_problem_difficulty d
WHERE d.difficulty IS NOT NULL
GROUP BY d.difficulty
ORDER BY min(d.rating)
```

Run with: `(1,)`

Output:

difficulty	total	solved
Easy	1790	240
Medium	1970	406
Hard	3383	448

52. My sheet progress

Purpose: Progress bars on the dashboard.

Concepts: LEFT JOIN with a VIEW

```
SELECT sh.sheet_id, sh.title, sh.deadline,
       COALESCE(vp.total, 0) AS total, COALESCE(vp.solved, 0) AS solved
FROM sheet sh
LEFT JOIN v_sheet_progress vp ON vp.sheet_id = sh.sheet_id AND vp.user_id = %s
WHERE sh.team_id = %s
ORDER BY sh.deadline NULLS LAST
```

Run with: `(1, 1)`

Output:

sheet_id	title	deadline	total	solved
1	Binary Search Week	2026-10-20	8	1

B7. Leaderboard

Source: `backend/routes/leaderboard.py`

53. Ranked leaderboard, optional college filter

Purpose: Students tied on solved count share a rank. Passing NULL for univ_id shows every college.

Concepts: VIEW, window function rank() OVER, optional filter with IS NULL OR, type cast

```
SELECT rank() OVER (ORDER BY lb.solved DESC) AS rank,
       lb.user_id, lb.name, lb.univ_name, t.team_name, lb.solved
FROM v_leaderboard lb
JOIN app_user u   ON u.user_id = lb.user_id
LEFT JOIN team t  ON t.team_id = u.team_id
WHERE %(univ_id)s::int IS NULL OR u.univ_id = %(univ_id)s::int
ORDER BY rank, lb.name
```

Run with: `{'univ_id': None}`

Output:

rank	user_id	name	univ_name	team_name	solved
1	2	Demo Student B	Amrita Vishwa Vidyapeetham	O(1)	4596
2	3	Demo Student C	Amrita Vishwa Vidyapeetham	O(1)	3956
3	4	Demo Student D	NIT Calicut	NULL	3049
4	1	Demo Student A	Amrita Vishwa Vidyapeetham	O(1)	1542

B8. Database page

Source: `backend/routes/database.py`

54. Row count of a table

Purpose: Run once per table. The table name comes from a fixed allow-list and is quoted with psycopg2.sql.Identifier, so it cannot be used for SQL injection.

Concepts: count(*), safe dynamic identifiers

```
SELECT count(*) AS n FROM {table}
```

Output (run with table = submission):

n
26944

55. First 500 rows of a table or view

Purpose: Shows live table contents on the Database page.

Concepts: SELECT *, LIMIT

```
SELECT * FROM {table} LIMIT 500
```

Output (run with table = university):

univ_id	univ_name	city
1	Amrita Vishwa Vidyapeetham	Amritapuri
2	NIT Calicut	Kozhikode

C. Analysis queries

Queries written to demonstrate SQL features on the project data (also in docs/lite/queries-lite.sql and the report).

C1. INNER JOIN of three tables: latest accepted submissions

```
SELECT u.name, p.problem_code, p.title, s.submitted_at
FROM submission s
JOIN app_user u ON u.user_id = s.user_id
JOIN problem  p ON p.problem_code = s.problem_code
WHERE s.verdict = 'OK'
ORDER BY s.submitted_at DESC
LIMIT 5;
```

Output:

name	problem_code	title	submitted_at
Demo Student A	2271H1	Personal Issues (Easy Version)	2026-10-10 18:38
Demo Student A	2271F	Beautiful Pillers	2026-10-10 17:21
Demo Student B	2271H1	Personal Issues (Easy Version)	2026-10-10 16:50
Demo Student B	2271F	Beautiful Pillers	2026-10-10 16:19
Demo Student A	2271E	Array Emptying	2026-10-10 16:12

C2. GROUP BY with HAVING: students with at least 2,000 problems solved

```
SELECT u.name, count(DISTINCT s.problem_code) AS solved
FROM app_user u JOIN submission s ON s.user_id = u.user_id
WHERE s.verdict = 'OK'
GROUP BY u.name
HAVING count(DISTINCT s.problem_code) >= 2000;
```

Output:

name	solved
Demo Student B	4596
Demo Student C	3956
Demo Student D	3049

C3. Correlated NOT EXISTS: sheet problems student 2 has not solved

```
SELECT sp.position, sp.problem_code
FROM sheet_problem sp
WHERE sp.sheet_id = 1
  AND NOT EXISTS (SELECT 1 FROM submission s
                  WHERE s.user_id = 2 AND s.problem_code = sp.problem_code
                    AND s.verdict = 'OK')
ORDER BY sp.position;
```

Output:

position	problem_code
1	2257D
2	2254E
3	2232C2
4	2229D
5	2228C2
6	2228C1

(6 of 8 rows shown)

C4. M:N join through the junction table: problems and average rating per topic

```
SELECT t.topic_name, count(*) AS problems, round(avg(p.rating)) AS avg_rating
FROM topic t
JOIN problem_topic pt ON pt.topic_id = t.topic_id
JOIN problem p        ON p.problem_code = pt.problem_code
GROUP BY t.topic_name
ORDER BY problems DESC, t.topic_name
LIMIT 6;
```

Output:

topic_name	problems	avg_rating
math	2370	1822
greedy	2337	1743
implementation	1803	1567
dp	1741	2262
constructive algorithms	1456	1876
data structures	1422	2277

C5. Window function rank() over a view

```
SELECT rank() OVER (ORDER BY solved DESC) AS rank, name, univ_name, solved
FROM v_leaderboard;
```

Output:

rank	name	univ_name	solved
1	Demo Student B	Amrita Vishwa Vidyapeetham	4596
2	Demo Student C	Amrita Vishwa Vidyapeetham	3956
3	Demo Student D	NIT Calicut	3049
4	Demo Student A	Amrita Vishwa Vidyapeetham	1542

C6. LEFT JOIN: every team with its member count

```
SELECT t.team_name, count(u.user_id) AS members
FROM team t
LEFT JOIN app_user u ON u.team_id = t.team_id
GROUP BY t.team_name
ORDER BY members DESC;
```

Output:

team_name	members
O(1)	3

C7. Aggregate inside a window (sum(count(*)) OVER ()): share of each verdict

```
SELECT verdict, count(*) AS submissions,
       round(100.0 * count(*) / sum(count(*)) OVER (), 1) AS percent
FROM submission
GROUP BY verdict
ORDER BY submissions DESC;
```

Output:

verdict	submissions	percent
OK	17326	64.3

verdict	submissions	percent
WRONG_ANSWER	6134	22.8
TIME_LIMIT_EXCEEDED	1823	6.8
RUNTIME_ERROR	1007	3.7
COMPILATION_ERROR	462	1.7
MEMORY_LIMIT_EXCEEDED	192	0.7

C8. Scalar subquery in WHERE: who solved the highest-rated problems

```
SELECT u.name, p.problem_code, p.title, p.rating
FROM submission s
JOIN app_user u ON u.user_id = s.user_id
JOIN problem p  ON p.problem_code = s.problem_code
WHERE s.verdict = 'OK'
  AND p.rating = (SELECT max(rating) FROM problem)
GROUP BY u.name, p.problem_code, p.title, p.rating
ORDER BY u.name, p.problem_code
LIMIT 5;
```

Output:

name	problem_code	title	rating
Demo Student A	750H	New Year and Snowy Grid	3500
Demo Student B	1246F	Cursor Distance	3500
Demo Student B	1292F	Nora's Toy Boxes	3500
Demo Student B	1305G	Kuroni and Antihype	3500
Demo Student B	1423C	Dušan's Railway	3500

C9. LEFT JOIN keeps colleges that have no students

```
SELECT un.univ_name, count(u.user_id) AS students
FROM university un
LEFT JOIN app_user u ON u.univ_id = un.univ_id
GROUP BY un.univ_name
ORDER BY students DESC;
```

Output:

univ_name	students
Amrita Vishwa Vidyapeetham	3
NIT Calicut	1

C10. Set operation INTERSECT: problems solved by both student 1 and student 2

```
SELECT p.problem_code, p.title
FROM problem p
WHERE p.problem_code IN (
    SELECT problem_code FROM submission WHERE user_id = 1 AND verdict = 'OK'
    INTERSECT
    SELECT problem_code FROM submission WHERE user_id = 2 AND verdict = 'OK')
ORDER BY p.rating DESC NULLS LAST
LIMIT 5;
```

Output:

problem_code	title
1523F	Favorite Game
1782G	Diverse Coloring
1446F	Line Distance
1349D	Slime and Biscuits
1142E	Pink Floyd

C11. Querying a view with ORDER BY ... NULLS LAST

```
SELECT * FROM v_problem_difficulty ORDER BY rating DESC NULLS LAST LIMIT 5;
```

Output:

problem_code	title	rating	difficulty
1427H	Prison Break	3500	Hard
1578G	Game of Chance	3500	Hard
1446E	Long Recovery	3500	Hard
1470F	Strange Covering	3500	Hard
750H	New Year and Snowy Grid	3500	Hard

C12. Querying the leaderboard view

```
SELECT * FROM v_leaderboard ORDER BY solved DESC;
```

Output:

user_id	name	univ_name	solved
2	Demo Student B	Amrita Vishwa Vidyapeetham	4596
3	Demo Student C	Amrita Vishwa Vidyapeetham	3956
4	Demo Student D	NIT Calicut	3049
1	Demo Student A	Amrita Vishwa Vidyapeetham	1542

C13. Querying the sheet-progress view

```
SELECT * FROM v_sheet_progress ORDER BY solved DESC;
```

Output:

sheet_id	user_id	name	total	solved
1	3	Demo Student C	8	3
1	1	Demo Student A	8	1
1	2	Demo Student B	8	0

D. Constraint and integrity tests

Each statement tries to break a rule (or checks a referential action). Every one was run on the demo database and rolled back. All 20 behaved as expected.

D1. Duplicate e-mail on sign-up

```
INSERT INTO app_user (name, email, password_hash, univ_id) VALUES ('X', 'a@demo.test', 'h', 1);
```

Expected: Rejected: UNIQUE(email)
Actual: Rejected: duplicate key value violates unique constraint "app_user_email_key" **PASS**

D2. Duplicate Codeforces handle

```
INSERT INTO app_user (name, email, password_hash, cf_handle, univ_id) VALUES ('X', 'x@demo.test', 'h', 'Errichto',
1);
```

Expected: Rejected: UNIQUE(cf_handle)
Actual: Rejected: duplicate key value violates unique constraint "app_user_cf_handle_key" **PASS**

D3. User at a university that does not exist

```
INSERT INTO app_user (name, email, password_hash, univ_id) VALUES ('X', 'x@demo.test', 'h', 999);
```

Expected: Rejected: foreign key
Actual: Rejected: insert or update on table "app_user" violates foreign key constraint "app_user_univ_id_fkey" **PASS**

D4. Missing name (NOT NULL)

```
INSERT INTO app_user (email, password_hash, univ_id) VALUES ('x@demo.test', 'h', 1);
```

Expected: Rejected: NOT NULL
Actual: Rejected: null value in column "name" of relation "app_user" violates not-null constraint **PASS**

D5. Problem rating out of range

```
INSERT INTO problem VALUES ('99999A', 'Too easy', 500);
```

Expected: Rejected: CHECK rating 800–3500
Actual: Rejected: new row for relation "problem" violates check constraint "problem_rating_check" **PASS**

D6. Unknown verdict

```
INSERT INTO submission VALUES (1, '4A', '2026-10-11 10:00', 'ACCEPTED', 'C++17');
```

Expected: Rejected: CHECK verdict
Actual: Rejected: new row for relation "submission" violates check constraint "submission_verdict_check" **PASS**

D7. Same submission twice (same user, problem, instant)

```
INSERT INTO submission SELECT user_id, problem_code, submitted_at, verdict, language FROM submission LIMIT 1;
```

Expected: Rejected: composite primary key
Actual: Rejected: duplicate key value violates unique constraint "submission_pkey" **PASS**

D8. Submission for a problem not in the database

```
INSERT INTO submission VALUES (1, '99999Z', '2026-10-11 10:00', 'OK', 'C++17');
```

Expected: Rejected: foreign key
Actual: Rejected: insert or update on table "submission" violates foreign key constraint "submission_problem_code_fkey" **PASS**

D9. Fourth member joins a full team

```
UPDATE app_user SET team_id = 1 WHERE user_id = 4;
```

Expected: Rejected by trigger team_size_check
Actual: Rejected: team 1 already has 3 members **PASS**

D10. Same problem twice on one sheet

```
INSERT INTO sheet_problem VALUES (1, '4A', 50);
```

Expected: Rejected: primary key (sheet_id, problem_code)
Actual: Rejected: duplicate key value violates unique constraint "sheet_problem_pkey" **PASS**

D11. Two problems at the same sheet position

```
INSERT INTO sheet_problem VALUES (1, '1A', 1);
```

Expected: Rejected: UNIQUE(sheet_id, position)
Actual: Rejected: duplicate key value violates unique constraint "sheet_problem_sheet_id_position_key" **PASS**

D12. Position zero on a sheet

```
INSERT INTO sheet_problem VALUES (1, '1A', 0);
```

Expected: Rejected: CHECK position > 0
Actual: Rejected: new row for relation "sheet_problem" violates check constraint "sheet_problem_position_check" **PASS**

D13. Duplicate team name

```
INSERT INTO team (team_name) VALUES ('O(1)');
```

Expected: Rejected: UNIQUE(team_name)
Actual: Rejected: duplicate key value violates unique constraint "team_team_name_key" **PASS**

D14. Deleting a sheet removes its problems (CASCADE)

```
DELETE FROM sheet WHERE sheet_id = 1;
SELECT count(*) AS sheet_problem_rows FROM sheet_problem WHERE sheet_id = 1;
```

Expected: Accepted; 0 sheet_problem rows left
Actual: Accepted; sheet_problem_rows = 0 **PASS**

D15. Deleting a team keeps its members (SET NULL)

```
DELETE FROM team WHERE team_id = 1;
SELECT count(*) AS users_left, count(team_id) AS users_with_team FROM app_user;
```

Expected: Accepted; users remain with team_id = NULL
Actual: Accepted; users_left = 4, users_with_team = 0 **PASS**

D16. Deleting a user removes their submissions (CASCADE)

```
DELETE FROM app_user WHERE user_id = 4;
SELECT count(*) AS submissions_of_user_4 FROM submission WHERE user_id = 4;
```

Expected: Accepted; 0 submissions left
Actual: Accepted; submissions_of_user_4 = 0 **PASS**

D17. Unrated problem is allowed (NULL passes CHECK)

```
INSERT INTO problem VALUES ('99999B', 'New unrated problem', NULL);
SELECT difficulty FROM v_problem_difficulty WHERE problem_code = '99999B';
```

Expected: Accepted; difficulty NULL
Actual: Accepted; difficulty = NULL **PASS**

D18. Re-importing a submission (sync twice)

```
INSERT INTO submission SELECT user_id, problem_code, submitted_at, verdict, language FROM submission LIMIT 1 ON
CONFLICT (user_id, problem_code, submitted_at) DO NOTHING;
```

Expected: Accepted; 0 rows inserted, no duplicate
Actual: Accepted; INSERT 0 0 **PASS**

D19. Fuzzy college search with a typo

```
SELECT univ_name, round(word_similarity('amritha vidya', univ_name)::numeric, 2) AS score FROM university WHERE
word_similarity('amritha vidya', univ_name) > 0.3;
```

Expected: Returns Amrita Vishwa Vidyapeetham
Actual: Accepted; univ_name = Amrita Vishwa Vidyapeetham, score = 0.48 **PASS**

D20. Leaving a team (nullable FK)

```
UPDATE app_user SET team_id = NULL WHERE user_id = 3;
SELECT name, team_id FROM app_user WHERE user_id = 3;
```

Expected: Accepted; team_id NULL
Actual: Accepted; name = Demo Student C, team_id = NULL **PASS**

E. Maintenance (backend/init_db.py and psql)

E1. Create the database if it is missing

```
SELECT 1 FROM pg_database WHERE datname = %s;
CREATE DATABASE "o1";
```

E2. Bring an older database up to date (safe to run again)

```
SELECT to_regclass('app_user') IS NOT NULL;   -- do the tables exist?

CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX IF NOT EXISTS university_name_trgm ON university USING gin (univ_name gin_trgm_ops);
ALTER TABLE app_user ADD COLUMN IF NOT EXISTS cf_rating INT;
ALTER TABLE app_user ADD COLUMN IF NOT EXISTS cf_max_rating INT;
ALTER TABLE app_user ALTER COLUMN password_hash TYPE VARCHAR(255);
ALTER TABLE submission ALTER COLUMN language TYPE VARCHAR(50);
CREATE OR REPLACE VIEW v_problem_difficulty AS
SELECT problem_code, title, rating,
       CASE WHEN rating IS NULL THEN NULL
            WHEN rating < 1400 THEN 'Easy'
            WHEN rating < 2000 THEN 'Medium'
            ELSE 'Hard' END AS difficulty
FROM problem;
```

E3. Reset: delete every table and start again (init_db.py --reset)

```
DROP SCHEMA public CASCADE;
CREATE SCHEMA public;
```

E4. Clear all data but keep the tables

```
TRUNCATE university, team, app_user, topic, problem, problem_topic,
         submission, sheet, sheet_problem RESTART IDENTITY CASCADE;
```

RESTART IDENTITY resets the SERIAL counters, so the next user, team and sheet start at id 1.

F. Index of SQL concepts

Concept	Where it is used
3-table JOIN	20, 48
Aggregate FILTER clause	45
Aggregate inside a window (sum(count(*)) OVER ())	C7
array_agg ... FILTER	17
BEFORE UPDATE trigger	39
bool_or	29
bool_or aggregate	17
CASE	17, 29
CHECK (rating BETWEEN 800 AND 3500)	14
COALESCE	17, 26, 33, 45
composite primary key	16
Correlated NOT EXISTS	C3
correlated scalar subqueries	41
count(*)	37, 47, 54
count(DISTINCT)	33, 36, 45
count(DISTINCT) FILTER	50
CTE	42, 43
date arithmetic	41
date_trunc	42
DELETE	32
derived table	46
dynamic WHERE with parameters	20
EXCLUDED pseudo-table	15
Existence check	7, 8
Existence check on primary key	19
EXISTS inside FILTER	51
FILTER	41
FK team_id	30
foreign key univ_id	9
Function in WHERE (lower)	5
gaps-and-islands	41
GIN trigram index	4
GROUP BY	17, 29, 33, 42, 43, 50, 51
GROUP BY with HAVING	C2
Idempotent INSERT ... ON CONFLICT (composite key) DO NOTHING RETURNING	24
ILIKE pattern matching	4
INNER JOIN	2
INNER JOIN of three tables	C1
INSERT ... RETURNING	6, 9, 30, 38
INSERT into a weak-entity table (composite key)	21
intervals	41
IS NOT NULL	44
JOIN	29, 36, 49
JOIN junction table + VIEW	27
JOIN through the M:N junction table	43
JOIN with a VIEW	17, 35

Concept	Where it is used
Junction table	16
Junction table with an attribute (position)	31
LEFT JOIN	33, 50, C6
LEFT JOIN (a user may have no team)	2
LEFT JOIN keeps colleges that have no students	C9
LEFT JOIN with a VIEW	26, 28, 52
LEFT JOINs	17
LIMIT	4, 20, 48, 49, 55
lower()	8
M	C4
NOT EXISTS	41
NOT EXISTS correlated subquery	48
NULLIF	41
NULLIF to avoid division by zero	45
NULLS LAST	17, 26
ON CONFLICT DO NOTHING	16
optional filter with IS NULL OR	53
ORDER BY	3, 18, 27
ORDER BY an aggregate	51
ORDER BY DESC	20, 49
ORDER BY on a boolean expression	4
pg_trgm similarity() and word_similarity()	4
projection	2
Querying a view with ORDER BY ... NULLS LAST	C11
Querying the leaderboard view	C12
Querying the sheet-progress view	C13
referential action CASCADE	32
round	45
safe dynamic identifiers	54
Scalar subquery in WHERE	C8
SELECT *	55
Selection	12, 22, 34
Selection by primary key	1
Selection on the alternate key email	10
Selection with two conditions	25
SERIAL key	6
Set operation INTERSECT	C10
string_agg	33
transaction (atomicity)	38
type cast	53
UNIQUE(topic_name)	15
UNIQUE(univ_name)	6
UPDATE	13, 23, 39
UPDATE to NULL (nullable FK)	40
UPDATE with WHERE on primary key	11
UPSERT: INSERT ... ON CONFLICT DO NOTHING	14
UPSERT: ON CONFLICT DO UPDATE	15
VIEW	51, 53
window function rank() OVER	53
Window function rank() over a view	C5

Concept	Where it is used
window function row_number() OVER (PARTITION BY ... ORDER BY)	41
Window functions rank() OVER and count(*) OVER ()	46
WITH (CTE)	41
CREATE TABLE with PRIMARY KEY / FOREIGN KEY / CHECK / UNIQUE	A2–A11
GIN index (pg_trgm)	A3
PL/pgSQL trigger	A12, D9
CREATE VIEW	A13
Referential actions CASCADE / SET NULL	D14–D16
TRUNCATE ... RESTART IDENTITY	E4
ALTER TABLE	E2